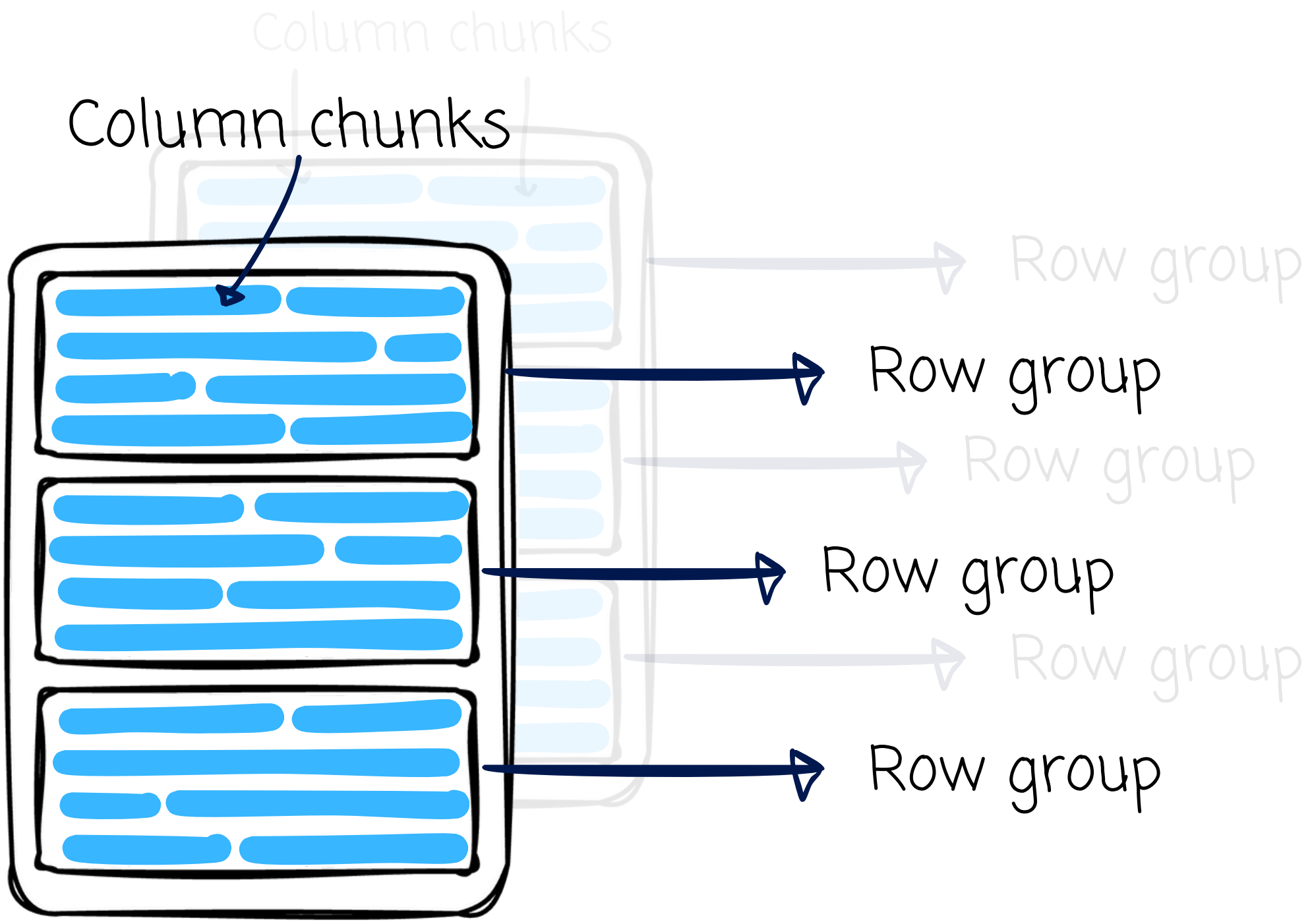
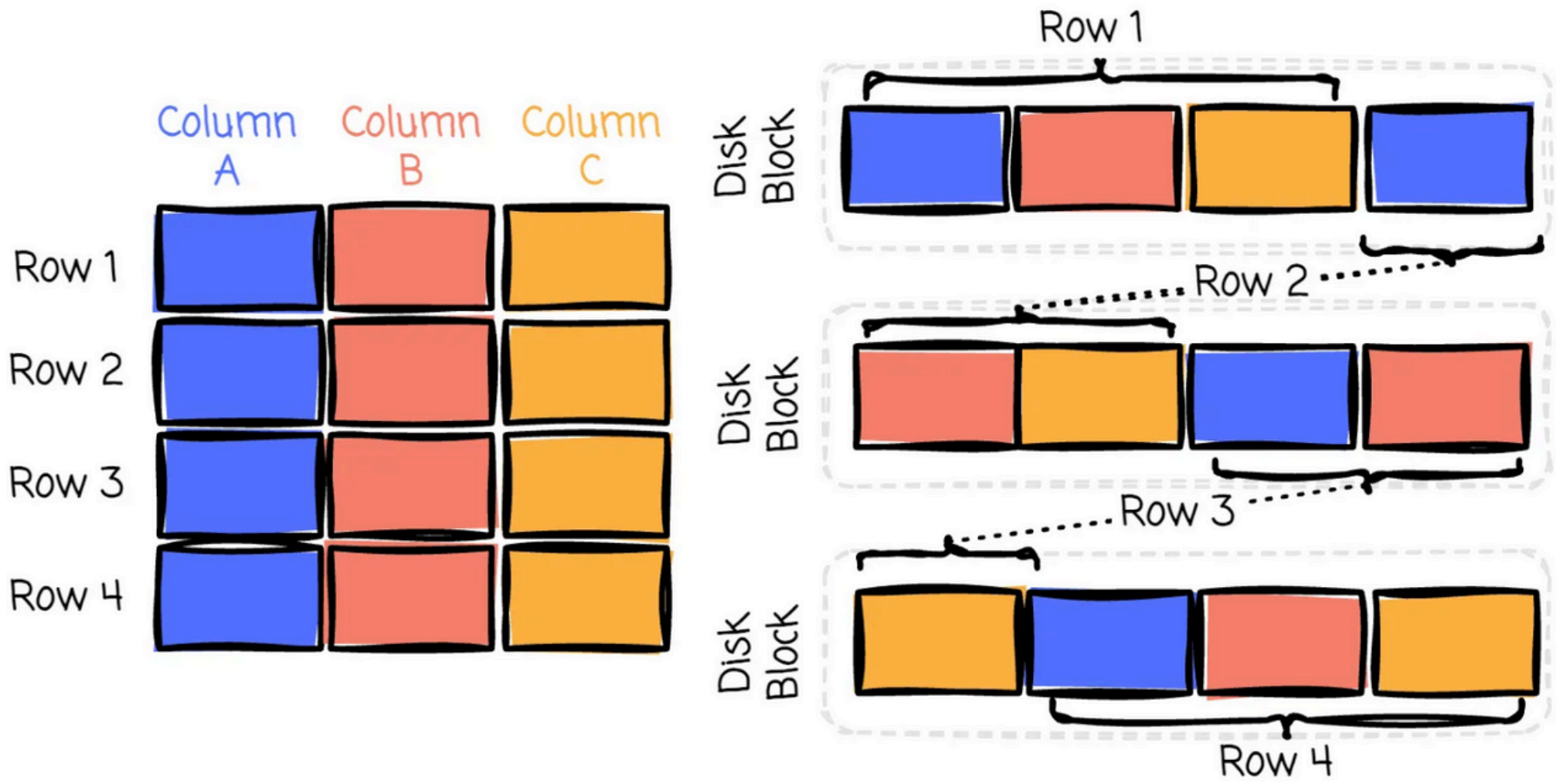


Parquet in 10 minutes

Kirill Bobrov x Vu Trinh

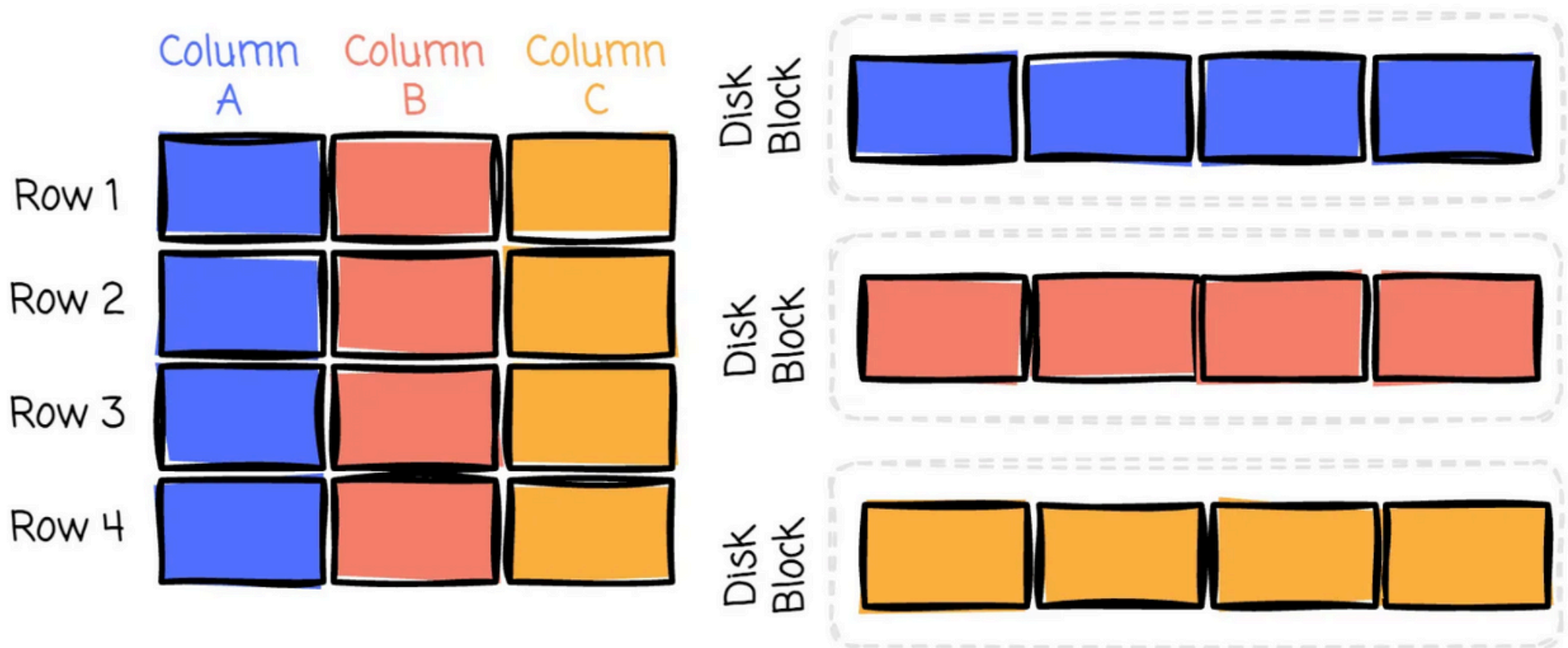
How does it organize data?

Traditional row-wise formats store data as records, one after another, much like a database table.



This format works well when accessing entire records frequently. However, it can be inefficient when dealing with analytics, where you often only need some columns from a large dataset.

Columnar formats stores data in columns instead of rows. This means that when you need specific columns, you can read only the data you need, reducing the amount of data scanned.

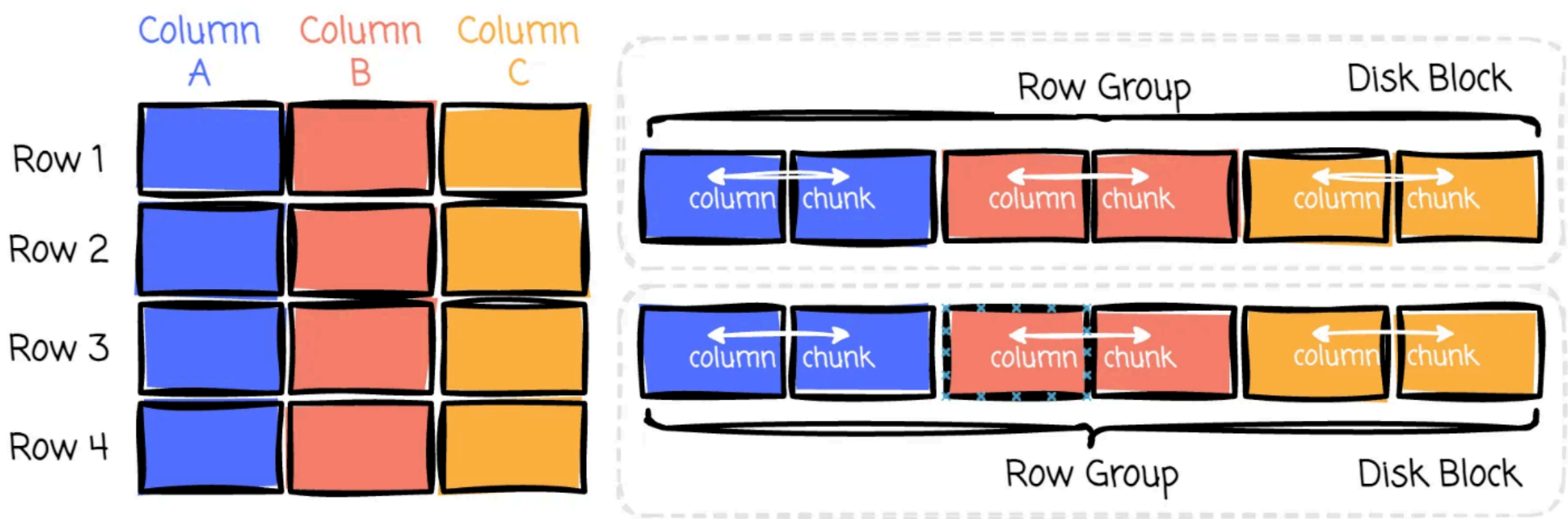


* in reality, the column might not fit entirely in the disk block; this image is only used for illustration. purpose

However, the record write or update operation requires touching multiple segments, which can slow the write performance.

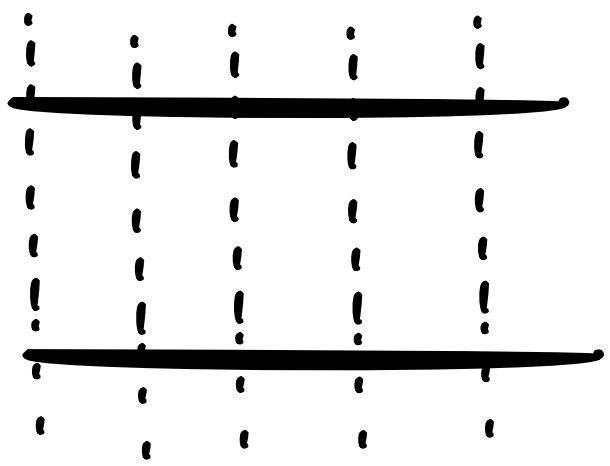
Plus, when queries involve multiple columns, it incurs the cost of reconstruct records from separate columns.

The hybrid format combines the best of both worlds.

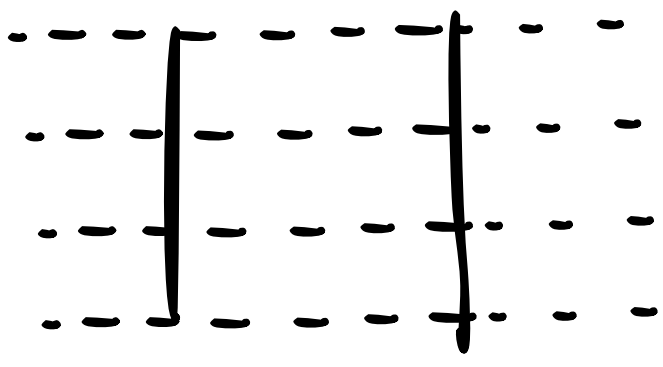


* in reality, the row group might not fit entirely in the disk block; this image is only used for illustration purpose.

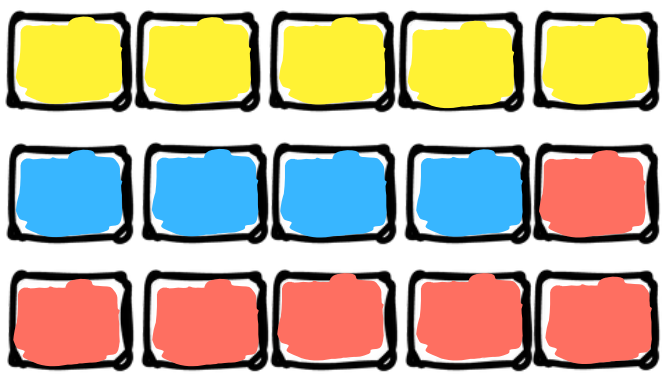
The format groups data into "row groups," each containing a subset of rows. (horizontal partition.)

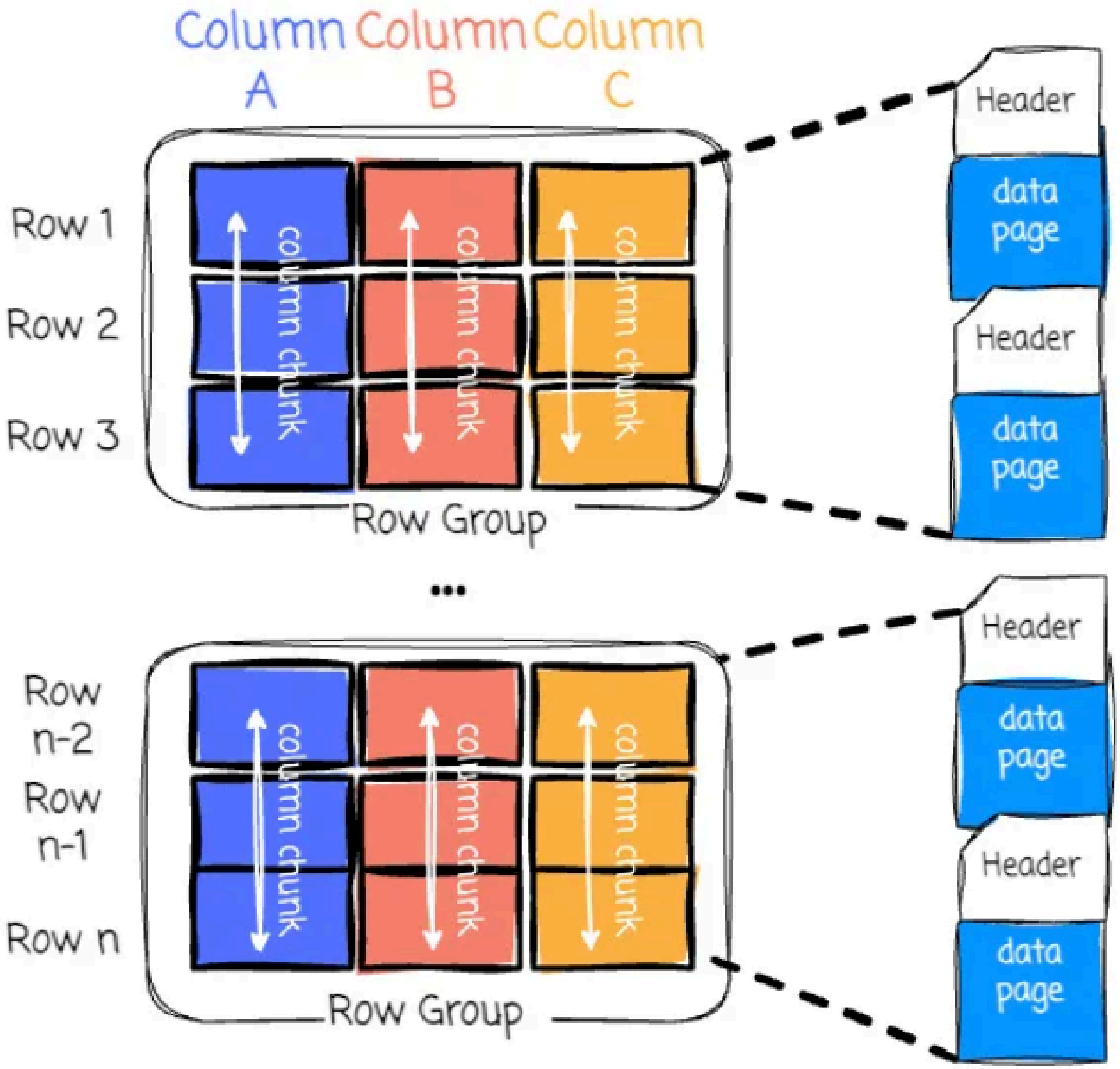


In each row group, data for each column is called a "column chunk." (vertical partition)



In the row group, these chunks are guaranteed to be stored contiguously on disk.





A Parquet file

A Parquet file is composed of:

Row Groups: Each row group contains a subset of the rows in the dataset.

Column Chunk: A chunk is the data for a particular column in the row group.

Pages: Column chunk is further divided into pages. A page is the smallest data unit in Parquet.

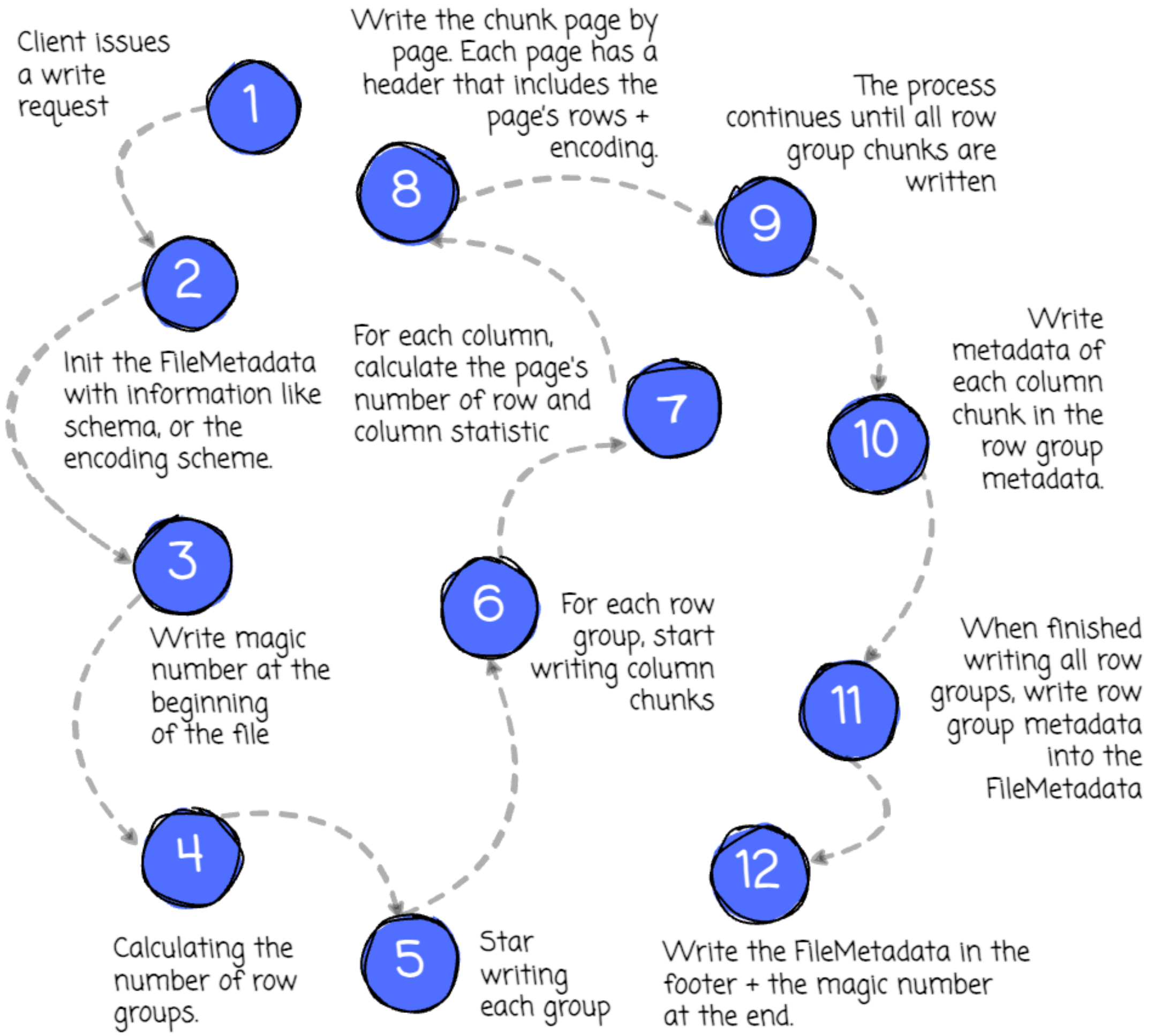
Parquet is a self-described file format; the metadata is the crucial part of Parquet

Magic number: A sequence of bytes (PAR1) located at the beginning and end of the file, used to verify if it is a valid Parquet file.

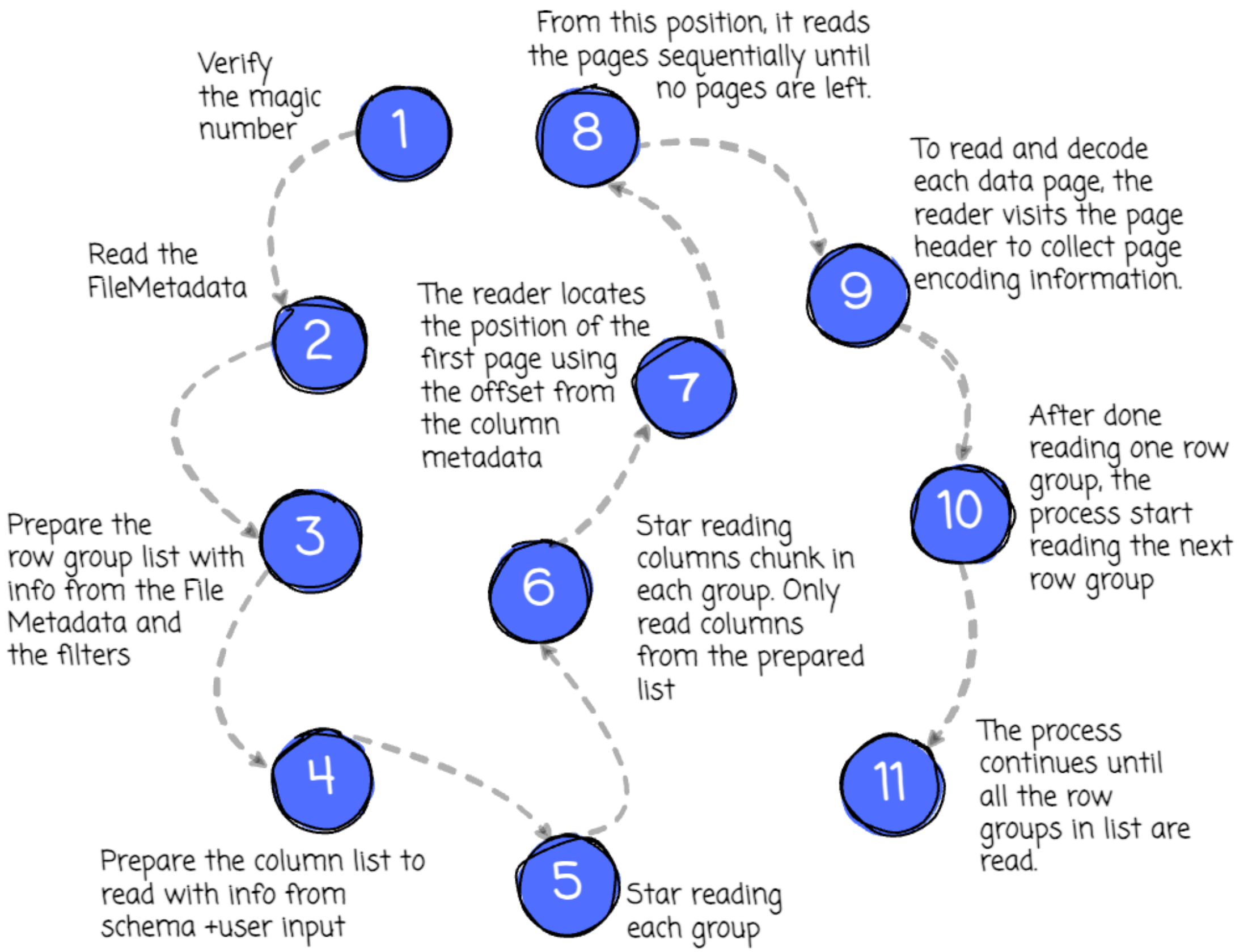
FileMetadata: Parquet stores FileMetadata in the footer of the file. This provides information like the number of rows, schema, and row group metadata.

PageHeader: The page header metadata is stored with the page data.

How is data written in the Parquet format?


Client issues a write request
1
2
Init the FileMetadata with information like schema, or the encoding scheme.
3
Write magic number at the beginning of the file
4
Calculating the number of row groups.
5
Star writing each group
6
For each row group, start writing column chunks
7
For each column, calculate the page's number of row and column statistic
8
Write the chunk page by page. Each page has a header that includes the page's rows + encoding.
9
The process continues until all row group chunks are written
10
Write metadata of each column chunk in the row group metadata.
11
When finished writing all row groups, write row group metadata into the FileMetadata
12
Write the FileMetadata in the footer + the magic number at the end.

How about the reading process?

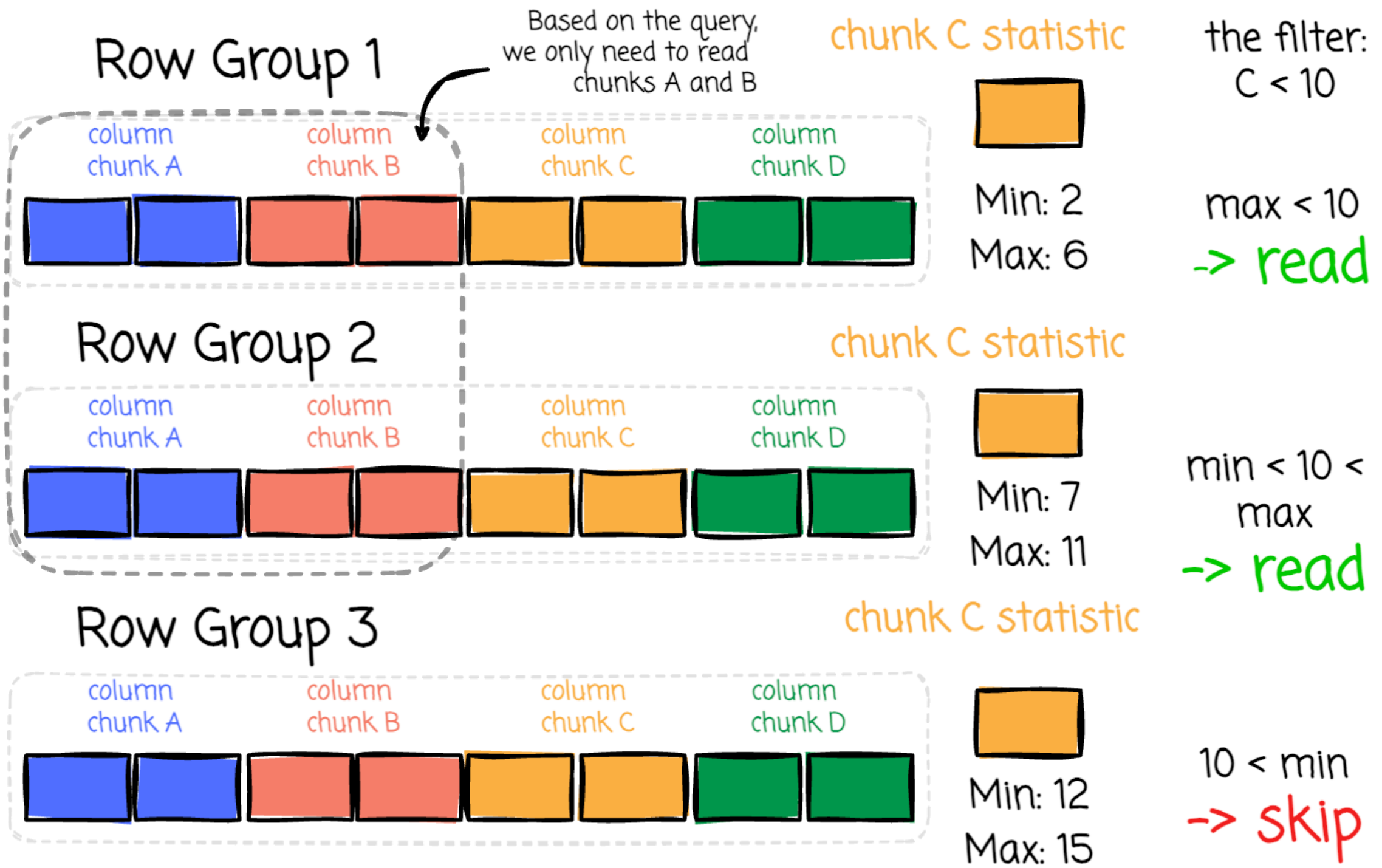

1
Verify the magic number
2
Read the FileMetadata
3
Prepare the row group list with info from the File Metadata and the filters
4
Prepare the column list to read with info from schema +user input
5
Star reading each group
6
Star reading columns chunk in each group. Only read columns from the prepared list
7
The reader locates the position of the first page using the offset from the column metadata
8
From this position, it reads the pages sequentially until no pages are left.
9
To read and decode each data page, the reader visits the page header to collect page encoding information.
10
After done reading one row group, the process start reading the next row group
11
The process continues until all the row groups in list are read.

How does it help with the OLAP workloads?

The ability to filter row groups using statistics and choose only the columns needed to read can significantly benefit the analytic workload. Giving the query:

```
SELECT column_A, SUM(column_B)
FROM my_table
WHERE column_C < 10
GROUP BY column_A
```

With the following Parquet layout, we only need to read row groups 1 and 2, focusing on columns A and B in each row group rather than reading all columns.


Row Group 1
Based on the query, we only need to read chunks A and B
column chunk A
column chunk B
column chunk C
column chunk D
chunk C statistic
Min: 2
Max: 6
the filter: C < 10
max < 10
-> read
Row Group 2
column chunk A
column chunk B
column chunk C
column chunk D
chunk C statistic
Min: 7
Max: 11
min < 10 < max
-> read
Row Group 3
column chunk A
column chunk B
column chunk C
column chunk D
chunk C statistic
Min: 12
Max: 15
10 < min
-> skip

Parquet Optimization & Best Practices

Leverage Predicate Pushdown

Predicate pushdown lets query engines filter data at the file or row group level.

Parquet stores min/max statistics for row group's each column.

Make sure your processing tools are configured to take advantage of this to skip unwanted data

Choosing the Right Compression Codec

Snappy: Fast compression/decompression with moderate compression ratio

Gzip: Higher compression ratio but slower

Zstandard (ZSTD): balance

Avoid Small Files

Too many small files →
metadata hell

Pick the Right Row Group Size

Larger: ↓ metadata overhead, ↑ read performance

Smaller: ↑ metadata overhead, ↑ parallelism

Optimize Encoding Strategies

Dictionary Encoding: 👍 for low-cardinality strings

Run-Length Encoding (RLE): 👍 for repeated values

Delta Encoding: 👍 for slowly incrementing numbers (e.g., timestamps).

Sort Data for Faster Queries

Sort by columns frequently used in WHERE clauses or range queries → predicate pushdown work more effectively + makes encodings more efficient.

Use Transactional Table Formats

Don't force ACID semantics on raw Parquet.

→ Apache Iceberg, Delta Lake, or Apache Hudi

If you like this, you'll surely enjoy the full article. You can find the link in the post or in the comments section.

Looking for high-quality data engineering content? Follow the authors' work on:

🚀 luminousmen.substack.com

🚀 vutr.substack.com